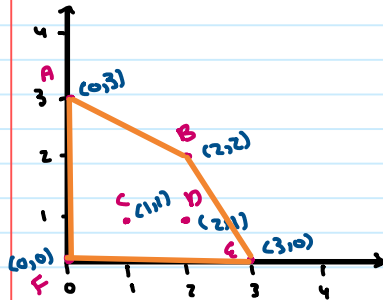


DAA ASSIGNMENT#03

Saturday, 8 November 2025 2:03 PM

NABIRA KHAN 23h-09m

QUESTION#01



starting point $\rightarrow (0,0)$

$$\text{orientation}(p, q, r) = (q_x - p_x) \cdot (r_y - p_y) - (q_y - p_y) \cdot (r_x - p_x)$$

STEP	CURRENT POINT (P)	CANDIDATE (Q)	CHECKING POINT (r)	ORIENTATION (P, Q, r)	MORE LOW?	NEW CANDIDATE
1	F(0,0)	A(0,3)	B(2,2)	-6	no	A
2	F(0,0)	A(0,3)	C(1,1)	-3	no	A
3	F(0,0)	A(0,3)	D(2,1)	-6	no	A
4	F(0,0)	A(0,3)	E(3,0)	-9	no	A
5	A(0,3)	B(2,2)	C(1,1)	-3	no	B
6	A(0,3)	B(2,2)	D(2,1)	-2	no	B
7	A(0,3)	B(2,2)	E(3,0)	-3	no	B
8	A(0,3)	B(2,2)	F(0,0)	-6	no	B
9	B(2,2)	C(1,1)	A(0,3)	-3	no	C
10	B(2,2)	C(1,1)	D(2,1)	1	yes	D
11	B(2,2)	D(2,1)	E(3,0)	1	yes	E
12	B(2,2)	E(3,0)	F(0,0)	-6	no	E
13	E(3,0)	A(0,3)	B(2,2)	-3	no	A
14	E(3,0)	A(0,3)	C(1,1)	3	yes	C
15	E(3,0)	C(1,1)	D(2,1)	-1	no	C
16	E(3,0)	C(1,1)	F(0,0)	3	yes	F

← reached F again

CONVEX HULL POINTS $\rightarrow \{F, A, B, E\}$

QUESTION#02

PART 1

BRUTE FORCE FAILURE FUNCTION ALGORITHM

for each position j:

- consider the substring $P[0..j]$
- for every possible prefix length k (from j down to 1):
 - compare prefix $P[0..k-1]$ with suffix $P[j-k+1..j]$
 - if they match, record $FF[j]=k$ and break
- if no match, set $FF[j]=0$

TIME COMPLEXITY

for each j, we might compare up to j prefixes & suffixes
worst case $\rightarrow T(n) = 1+2+3+\dots+(n-1) = O(n^2)$

$$T(n) = O(n^2)$$

PART 2

KMP OPTIMIZED FAILURE FUNCTION ALGORITHM

```

F[0] ← 0
i ← 1
j ← 0
while i < m
  if P[i] = P[j]
    F[i] ← j+1
    i ← i+1
    j ← j+1
  else if j > 0 then
    j ← F[j]-1
  else
    F[i] ← 0
    i ← i+1
  
```

When a mismatch occurs at position i , the algorithm doesn't restart comparisons from the beginning. Instead, it uses the previously computed value $F[j-1]$ to jump to the next possible prefix length that could still match. This reuse of $F[j]$ avoids rechecking characters, making the algorithm linear-time.

TIME COMPLEXITY

Each character is compared at most twice (forward or fall back)

$$T(n) = O(n)$$

PART 3

BRUTE FORCE

j	substring P[0:j]	LONGEST PROPER PREFIX = SUFFIX	F[j]
0	a	—	0
1	ab	—	0
2	aba	a	1
3	abab	ab	2
4	ababa	aba	3
5	ababac	—	0
6	ababaca	a	1

$$F = [0, 0, 1, 2, 3, 0, 1]$$

KMP OPTIMIZED

i	P[i]	j	COMPARISON	ACTION	F[i]
0	a	-	-	$F[0] = 0$	0
1	b	0	$P[1] \neq P[0]$	$F[1] = 0$	0
2	a	0	$P[2] = P[0]$	$F[2] = 1, j=1$	1
3	b	1	$P[3] = P[1]$	$F[3] = 2, j=2$	2
4	a	2	$P[4] = P[2]$	$F[4] = 3, j=3$	3
5	c	3	$P[5] \neq P[3] \rightarrow j = F[2] = 1$ $\rightarrow P[5] \neq P[1] \rightarrow j = F[0] = 0$	$F[5] = 0$	0
6	a	0	$P[6] = P[0]$	$F[6] = 1, j=1$	1

$$F = [0, 0, 1, 2, 3, 0, 1]$$

Both algorithms produce the same result

PART 4

FEATURE	BRUTE FORCE	KMP (OPTIMIZED)
TIME COMPLEXITY	$O(n^2)$	$O(n)$
COMPARISONS	many repeated checks	each char checked at most twice
REUSE INFO	none	uses previous F[i-1] values to skip redundant comparisons

QUESTION #03

PART A

COIN CHANGE PROBLEM

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1, 5	1	1	1	1	1	2	2	2	2	2	3	3	3	3
1, 5, 6	1	1	1	1	1	2	3	3	3	3	4	5	6	6
1, 5, 6, 8	1	1	1	1	1	2	3	3	4	4	5	6	7	8

8 possibilities total for target = 13

PART B

THE EDIT DISTANCE PROBLEM

str1 = "KITTEN" str2 = "SITTING"

	O	S	I	T	T	I	N	G
O	0	1	2	3	4	5	6	7
K	1	1	2	3	4	5	6	7
I	2	2	1	2	3	4	5	6
T	3	3	2	1	2	3	4	5
T	4	4	3	2	1	2	3	4
E	5	5	4	3	2	2	3	4
N	6	6	5	4	3	3	2	3

3 operations in total to convert "kitten" to "sitting"

PART C

THE ROD CUTTING PROBLEM

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	1	0	1	2	3	4	5	6	8
5	2	0	1	5	6	10	11	15	20
8	3	0	1	5	8	5	13	16	21
9	4	0	1	5	8	9	13	18	21
6	5								
16	6								
18	7								
20	8								

price = 21

PART D

SPACE-SEPARATED SEQUENCE

substring	break?	how?
" "	yes	empty
"i"	yes	i
"il"	no	-
"ili"	no	-
"ilik"	no	-
"i like"	yes	i + like
"i likea"	no	-
"i likeap"	no	-
"i likeapp"	no	-
"i likeappl"	no	-
"i likeapple"	yes	i + like + apple

∴ yes, it can be segmented

QUESTION #04

COURSE REGISTRATION PROBLEM

COURSE	STUDY HOURS	CREDITS
C1	2	4
C2	3	5
C3	4	7
C4	5	9
C5	6	11

TOTAL WEEKLY LIMIT = 10 hours

	0	2	3	4	5	6	7	8	9	10
C1	0	4	4	4	4	4	4	4	4	4
C2	0	4	5	5	5	9	9	9	9	9
C3	0	4	5	5	9	11	12	12	16	16
C4	0	4	5	5	9	11	13	14	16	18
C5	0	4	5	5	9	11	13	15	16	18

max credits = 18

courses selected = C2, C4, C1

total time = 10 hours

ALGORITHM

CourseSelectionDP(Credits[], Hours[], Limit)

n ← num of courses

create table DP[n+1][limit+1]

for i = 0 to n:

for t = 0 to limit:

if i = 0 or t = 0:

DP[i][t] ← 0

else if Hours[i-1] ≤ t:

DP[i][t] ← max(Credits[i-1] + DP[i-1][t - Hours[i-1]], DP[i-1][t])

else:

DP[i][t] ← DP[i-1][t]

return DP[n][limit]